



Mémoire: QUIC: A UDP-Based Multiplexed and Secure Transport draft-ietf-quic-transport-18



Academic year : 2020 - 2021

Teacher : BONAVENTURE Olivier,
LEGAY Axel
Course : Mémoire
Collaborators :
CROCHET Christophe,
Jean-François Sambon

1 Streams

- Streams are identified within a connection by a numeric value, referred to as the stream ID. A stream ID is a 62-bit integer (0 to 2⁶²-1) that is unique for all streams on a connection.

Bits	Stream Type
0x0	Client-Initiated, Bidirectional
0x1	Server-Initiated, Bidirectional
0x2	Client-Initiated, Unidirectional
0x3	Server-Initiated, Unidirectional

Table 1: Stream ID Types

2 Variable-Length Integer Encoding

QUIC packets and frames commonly use a variable-length encoding for non-negative integer values. This encoding ensures that smaller integer values need fewer bytes to encode.

The QUIC variable-length integer encoding reserves the two most significant bits of the first byte to encode the base 2 logarithm of the integer encoding length in bytes. The integer value is encoded on the remaining bits, in network byte order.

This means that integers are encoded on 1, 2, 4, or 8 bytes and can encode 6, 14, 30, or 62 bit values respectively. Table 4 summarizes the encoding properties.

2Bit	Length	Usable Bits	Range
00	1	6	0-63
01	2	14	0-16383
10	4	30	0-1073741823
11	8	62	0-4611686018427387903

Table 4: Summary of Integer Encodings

3 Packets

3.1 Long Header Packets

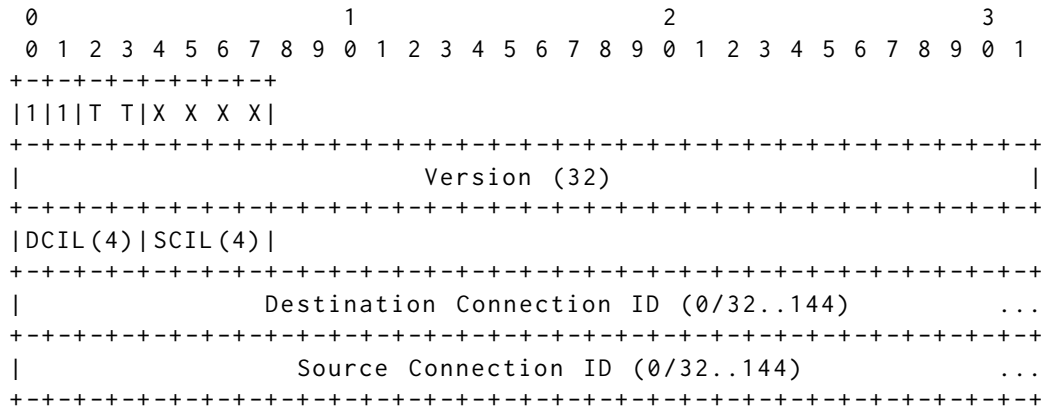


Figure 10: Long Header Packet Format

Type	Name	Section
0x0	Initial	Section 17.2.2
0x1	0-RTT	Section 17.2.3
0x2	Handshake	Section 17.2.4
0x3	Retry	Section 17.2.5

Table 5: Long Header Packet Types

3.1.1 Version Negotiation Packet

The Version field of a Version Negotiation packet MUST be set to 0x00000000.

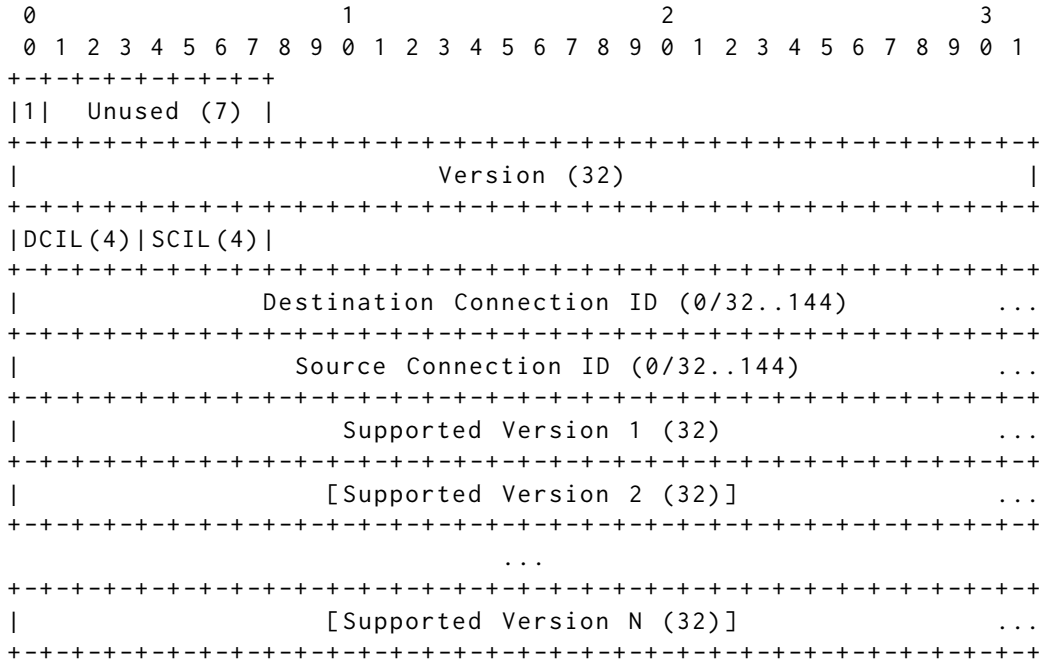
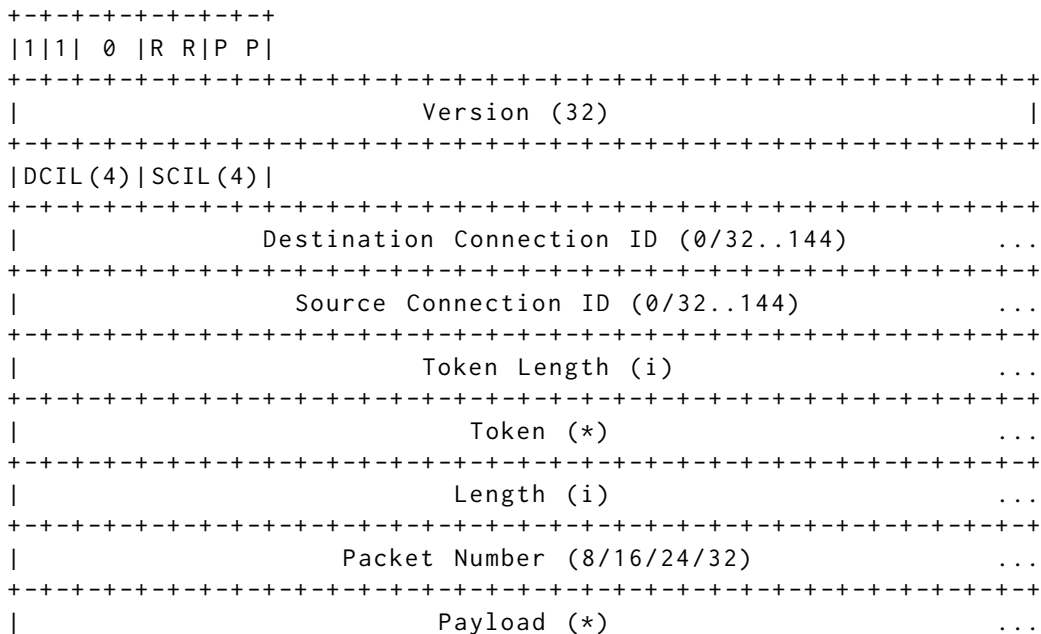


Figure 11: Version Negotiation Packet

3.1.2 Initial Packet



```

+-----+

```

Figure 12: Initial Packet

3.1.3 0-RTT

```

+-----+
|1|1| 1 |R R|P P|
+-----+
|                                     Version (32)                                     |
+-----+
|DCIL (4)|SCIL (4)|
+-----+
|          Destination Connection ID (0/32..144)          ...
+-----+
|          Source Connection ID (0/32..144)                ...
+-----+
|          Length (i)                                     ...
+-----+
|          Packet Number (8/16/24/32)                     ...
+-----+
|          Payload (*)                                     ...
+-----+

```

0-RTT Packet

3.1.4 Handshake Packet

```

+-----+
|1|1| 2 |R R|P P|
+-----+
|                                     Version (32)                                     |
+-----+
|DCIL (4)|SCIL (4)|
+-----+
|          Destination Connection ID (0/32..144)          ...
+-----+
|          Source Connection ID (0/32..144)                ...
+-----+
|          Length (i)                                     ...
+-----+
|          Packet Number (8/16/24/32)                     ...
+-----+
|          Payload (*)                                     ...
+-----+

```

Figure 13: Handshake Protected Packet

3.1.5 Retry Packet

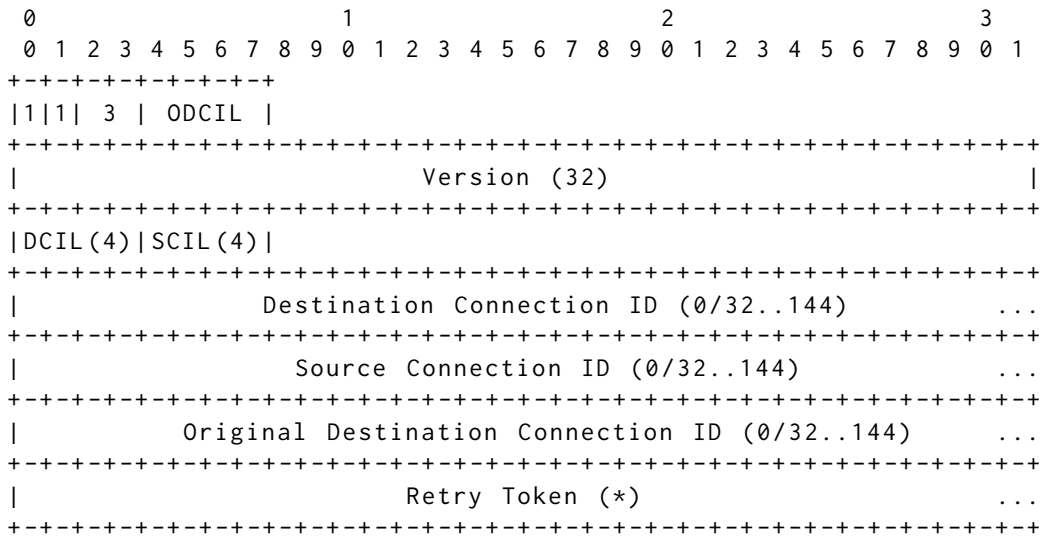


Figure 14: Retry Packet

3.2 Short Header Packets

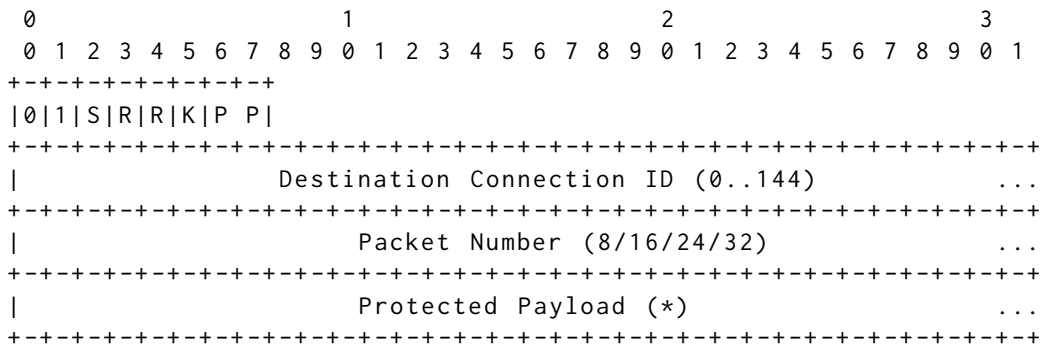
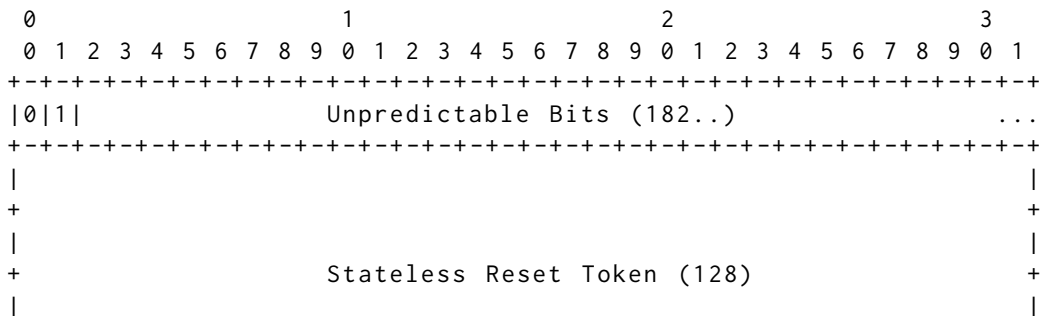


Figure 15: Short Header Packet Format

3.3 Stateless Reset Packet



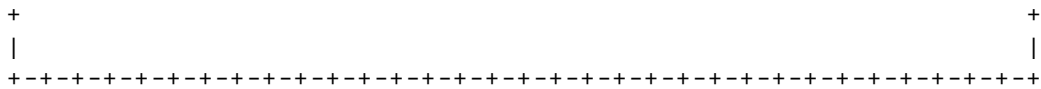


Figure 7: Stateless Reset Packet

4 Transport Parameter Encoding

```

uint32 QuicVersion;

enum {
    original_connection_id(0),
    idle_timeout(1),
    stateless_reset_token(2),
    max_packet_size(3),
    initial_max_data(4),
    initial_max_stream_data_bidi_local(5),
    initial_max_stream_data_bidi_remote(6),
    initial_max_stream_data_uni(7),
    initial_max_streams_bidi(8),
    initial_max_streams_uni(9),
    ack_delay_exponent(10),
    max_ack_delay(11),
    disable_migration(12),
    preferred_address(13),
    (65535)
} TransportParameterId;

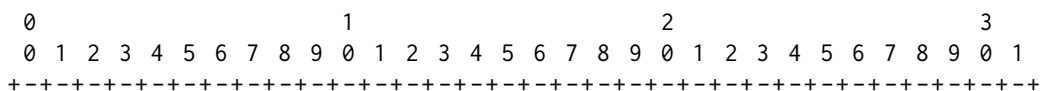
struct {
    TransportParameterId parameter;
    opaque value<0..2^16-1>;
} TransportParameter;

struct {
    select (Handshake.msg_type) {
        case client_hello:
            QuicVersion initial_version;

            case encrypted_extensions:
                QuicVersion negotiated_version;
                QuicVersion supported_versions<4..2^8-4>;
    };
    TransportParameter parameters<0..2^16-1>;
} TransportParameters;

```

Figure 16: Definition of TransportParameters



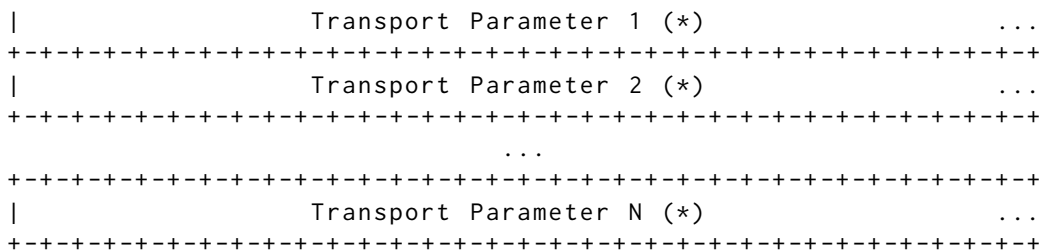


Figure 16: Sequence of Transport Parameters

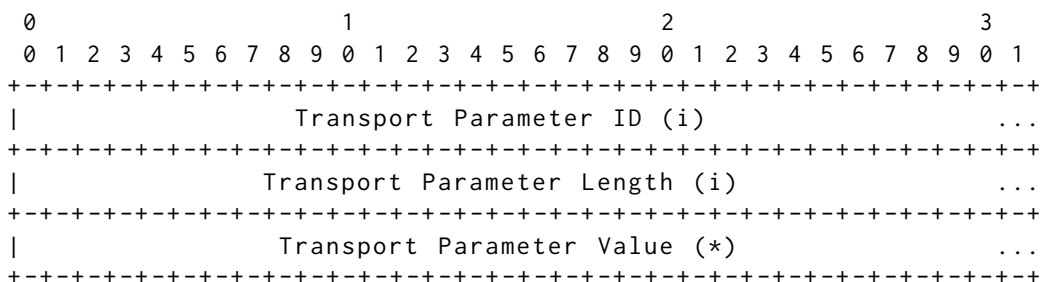


Figure 17: Transport Parameter Encoding

`original_connection_id` (0x0000): The value of the Destination Connection ID field from the first Initial packet sent by the client. This transport parameter is only sent by a server. A server MUST include the `original_connection_id` transport parameter **if** it sent a Retry packet.

`idle_timeout` (0x0001): The idle timeout is a value in seconds that is encoded as an integer, see (Section 10.2). If **this** parameter is absent or zero **then** the idle timeout is disabled.

`stateless_reset_token` (0x0002): A stateless reset token is used in verifying a stateless reset, see Section 10.4. This parameter is a sequence of 16 bytes. This transport parameter is only sent by a server.

`max_packet_size` (0x0003): The maximum packet size parameter is an integer value that limits the size of packets that the endpoint is willing to receive. This indicates that packets larger than **this** limit will be dropped. The **default for this** parameter is the maximum permitted UDP payload of 65527. Values below 1200 are invalid. This limit only applies to **protected** packets (Section 12.1).

`initial_max_data` (0x0004): The initial maximum data parameter is an integer value that contains the initial value **for** the maximum amount of data that can be sent on the connection. This is

equivalent to sending a MAX_DATA (Section 19.9) **for** the connection immediately after completing the handshake.

`initial_max_stream_data_bidi_local (0x0005)`: This parameter is an integer value specifying the initial flow control limit **for** locally-initiated bidirectional streams. This limit applies to newly created bidirectional streams opened by the endpoint that sends the transport parameter. In client transport parameters, **this** applies to streams with an identifier with the least significant two bits set to 0x0; in server transport parameters, **this** applies to streams with the least significant two bits set to 0x1.

`initial_max_stream_data_bidi_remote (0x0006)`: This parameter is an integer value specifying the initial flow control limit **for** peer-initiated bidirectional streams. This limit applies to newly created bidirectional streams opened by the endpoint that receives the transport parameter. In client transport parameters, **this** applies to streams with an identifier with the least significant two bits set to 0x1; in server transport parameters, **this** applies to streams with the least significant two bits set to 0x0.

`initial_max_stream_data_uni (0x0007)`: This parameter is an integer value specifying the initial flow control limit **for** unidirectional streams. This limit applies to newly created unidirectional streams opened by the endpoint that receives the transport parameter. In client transport parameters, **this** applies to streams with an identifier with the least significant two bits set to 0x3; in server transport parameters, **this** applies to streams with the least significant two bits set to 0x2.

`initial_max_streams_bidi (0x0008)`: The initial maximum bidirectional streams parameter is an integer value that contains the initial maximum number of bidirectional streams the peer may initiate. If **this** parameter is absent or zero, the peer cannot open bidirectional streams until a MAX_STREAMS frame is sent. Setting **this** parameter is equivalent to sending a MAX_STREAMS (Section 19.11) of the corresponding **type** with the same value.

`initial_max_streams_uni (0x0009)`: The initial maximum unidirectional streams parameter is an integer value that contains the initial maximum number of unidirectional streams the peer may initiate. If **this** parameter is absent or zero, the peer cannot open unidirectional streams until a MAX_STREAMS frame is sent. Setting **this** parameter is equivalent to sending a MAX_STREAMS (Section 19.11) of the corresponding **type** with the same value.

`ack_delay_exponent (0x000a)`: The ACK delay exponent is an integer value indicating an exponent used to decode the ACK Delay field in the ACK frame (Section 19.3). If **this** value is absent, a **default** value of 3 is assumed (indicating a multiplier of 8). The **default**

value is also used for ACK frames that are sent in Initial and Handshake packets. Values above 20 are invalid.

`max_ack_delay (0x000b)`: The maximum ACK delay is an integer value indicating the maximum amount of time in milliseconds by which the endpoint will delay sending acknowledgments. This value SHOULD include the receiver's expected delays in alarms firing. For

example, if a receiver sets a timer for 5ms and alarms commonly fire up to 1ms late, then it should send a `max_ack_delay` of 6ms. If this value is absent, a default of 25 milliseconds is assumed. Values of 2^{14} or greater are invalid.

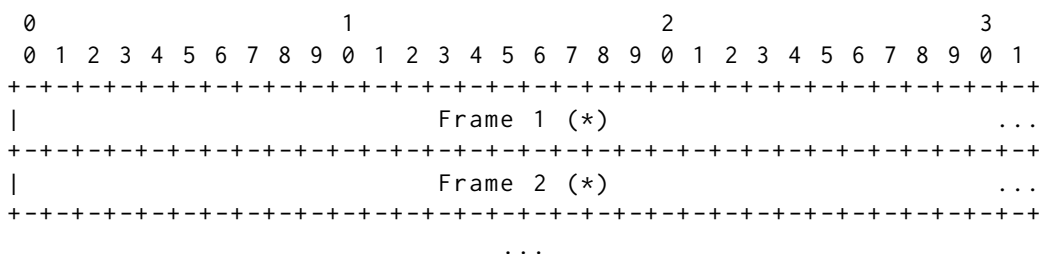
`disable_migration (0x000c)`: The disable migration transport parameter is included if the endpoint does not support connection migration (Section 9). Peers of an endpoint that sets this transport parameter MUST NOT send any packets, including probing packets (Section 9.1), from a local address other than that used to perform the handshake. This parameter is a zero-length value.

`preferred_address (0x000d)`: The server's preferred address is used to effect a change in server address at the end of the handshake, as described in Section 9.6. The format of this transport parameter is the PreferredAddress struct shown in Figure 17. This transport parameter is only sent by a server. Servers MAY choose to only send a preferred address of one address family by sending an all-zero address and port (0.0.0.0:0 or ::.0) for the other family.

```
struct {
    opaque ipv4Address[4];
    uint16 ipv4Port;
    opaque ipv6Address[16];
    uint16 ipv6Port;
    opaque connectionId<0..18>;
    opaque statelessResetToken[16];
} PreferredAddress;
```

Figure 17: Preferred Address format

5 Frames



```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                                     Frame N (*)                                     ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 7: QUIC Payload

```

0                                     1                                     2                                     3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                                     Frame Type (i)                                     ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                                     Type-Dependent Fields (*)                                     ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 8: Generic Frame Layout

Type	Value	Frame Type Name	Definition
0x00		PADDING	Section 19.1
0x01		PING	Section 19.2
0x02 - 0x03		ACK	Section 19.3
0x04		RESET_STREAM	Section 19.4
0x05		STOP_SENDING	Section 19.5
0x06		CRYPTO	Section 19.6
0x07		NEW_TOKEN	Section 19.7
0x08 - 0x0f		STREAM	Section 19.8
0x10		MAX_DATA	Section 19.9
0x11		MAX_STREAM_DATA	Section 19.10
0x12 - 0x13		MAX_STREAMS	Section 19.11
0x14		DATA_BLOCKED	Section 19.12
0x15		STREAM_DATA_BLOCKED	Section 19.13
0x16 - 0x17		STREAMS_BLOCKED	Section 19.14
0x18		NEW_CONNECTION_ID	Section 19.15

0x19	RETIRE_CONNECTION_ID	Section 19.16	
0x1a	PATH_CHALLENGE	Section 19.17	
0x1b	PATH_RESPONSE	Section 19.18	
0x1c - 0x1d	CONNECTION_CLOSE	Section 19.19	
+-----+			

Table 3: Frame Types

I: Initial (Section 17.2.2)

H: Handshake (Section 17.2.4)

0: 0-RTT (Section 17.2.3)

1: 1-RTT (Section 17.3)

*: A CONNECTION_CLOSE frame of type 0x1c can appear in Initial, Handshake, and 1-RTT packets, whereas a CONNECTION_CLOSE of type 0x1d can only appear in a 1-RTT packet.

5.1 ACK Frames

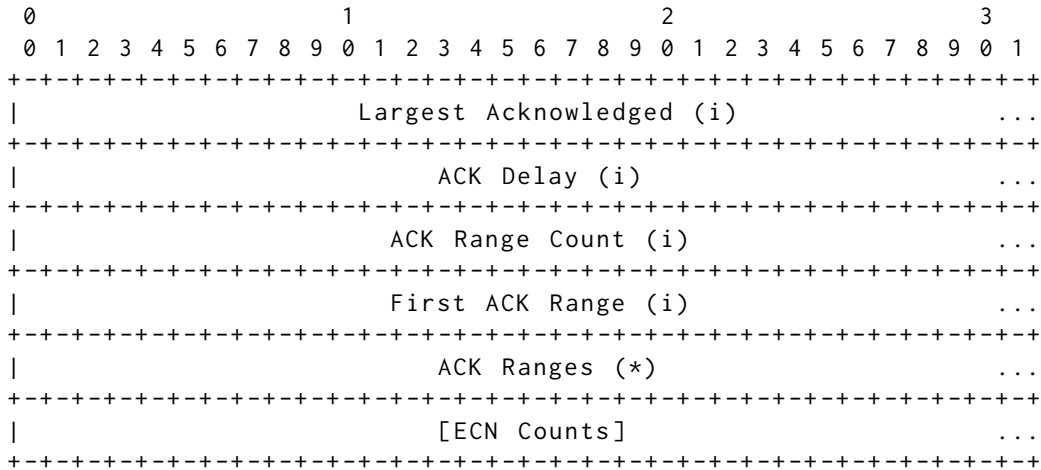


Figure 18: ACK Frame Format

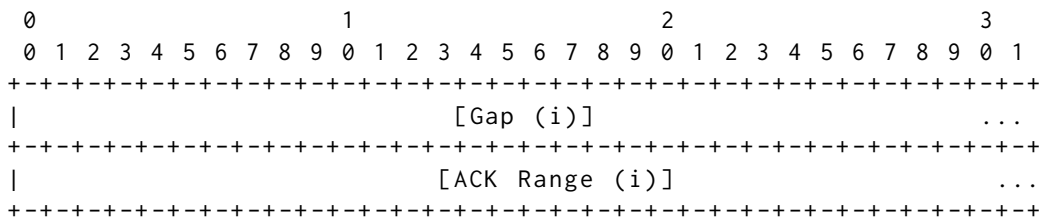




Figure 19: ACK Ranges

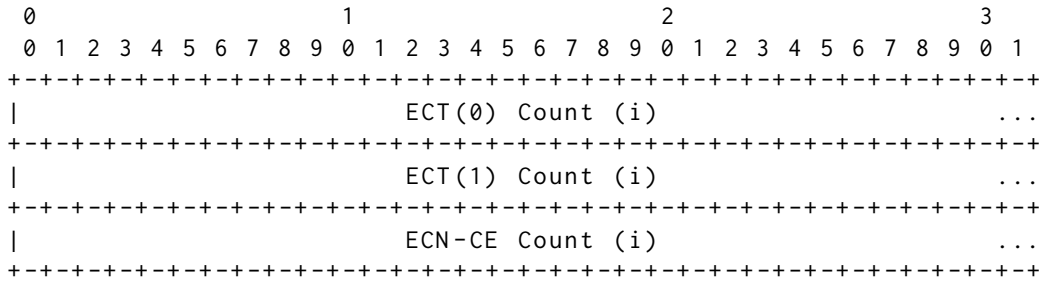


Figure 21: ECN Count Format

5.1.1 RESET_STREAM Frame

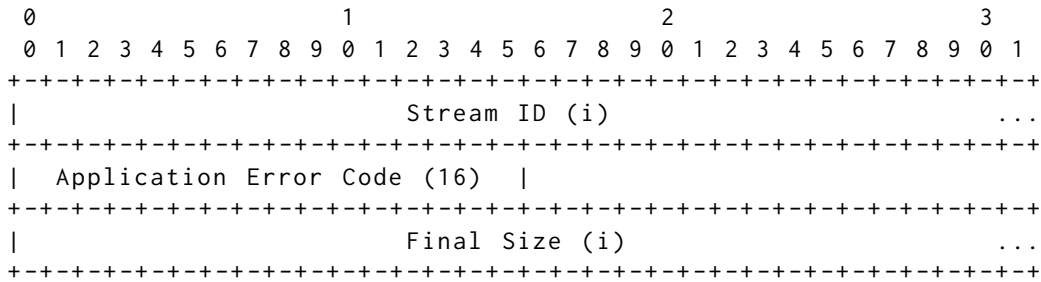


Figure 22: RESET_STREAM Frame Format

5.1.2 STOP_SENDING Frame

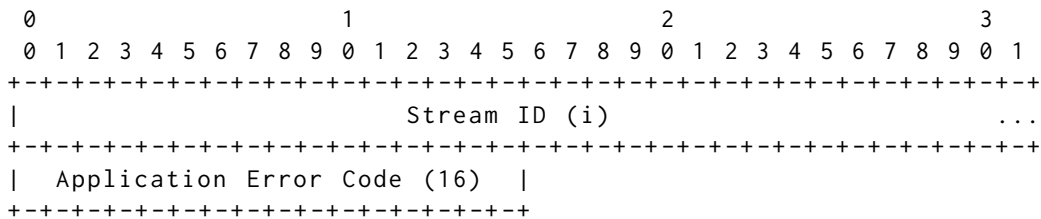


Figure 23: STOP_SENDING Frame Format

5.1.3 CRYPTO Frames

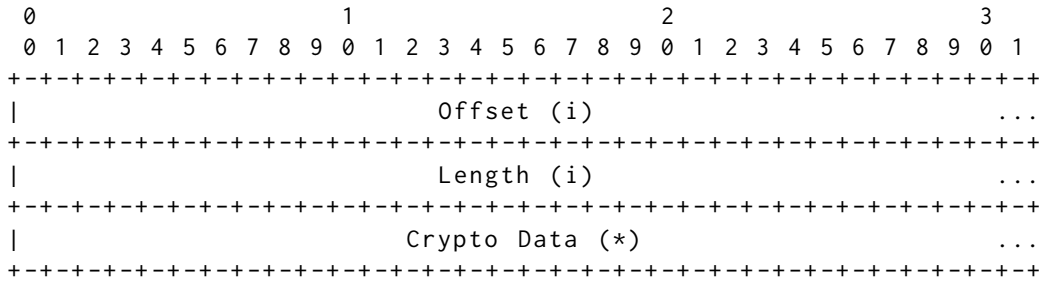


Figure 24: CRYPTO Frame Format

5.1.4 NEW_TOKEN Frame

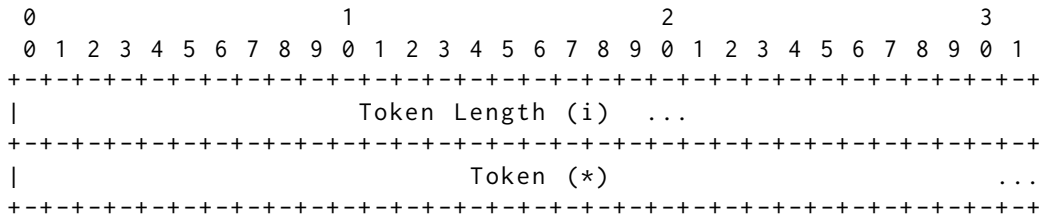


Figure 25: NEW_TOKEN Frame Format

5.1.5 STREAM Frames

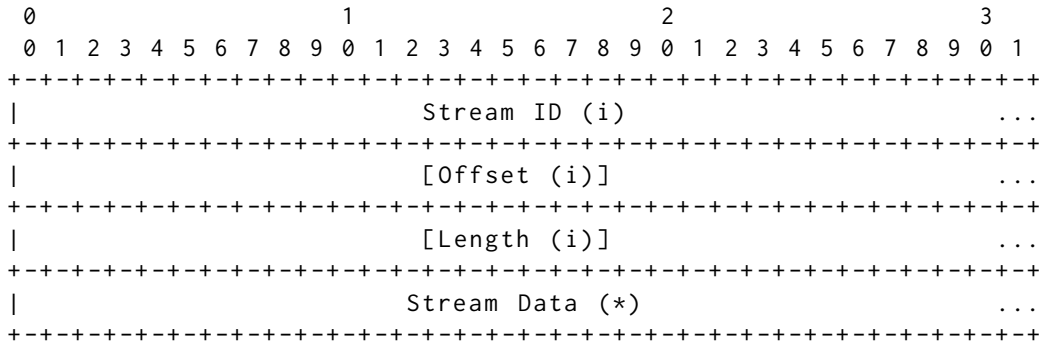


Figure 26: STREAM Frame Format

5.1.6 MAX_DATA Frame

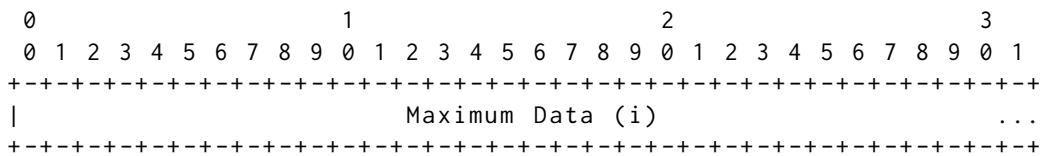


Figure 27: MAX_DATA Frame Format

5.1.7 MAX_STREAM_DATA Frame

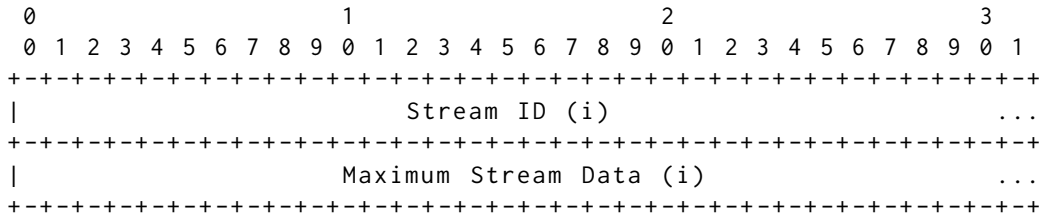


Figure 28: MAX_STREAM_DATA Frame Format

5.1.8 MAX_STREAMS Frames

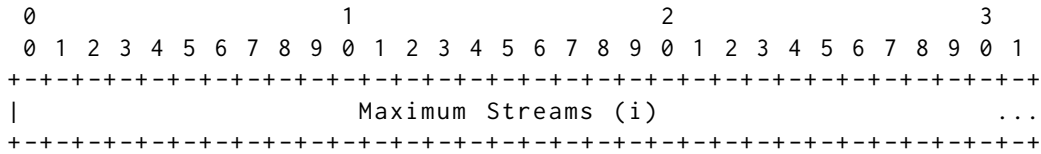


Figure 29: MAX_STREAMS Frame Format

5.1.9 DATA_BLOCKED Frame

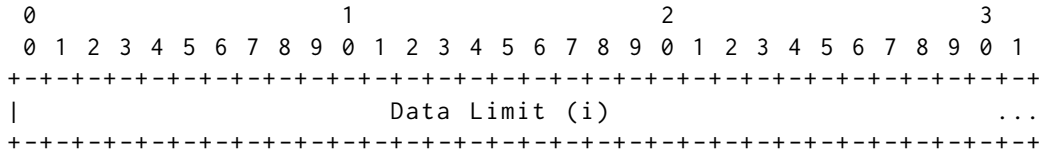


Figure 30: DATA_BLOCKED Frame Format

5.1.10 STREAM_DATA_BLOCKED Frame

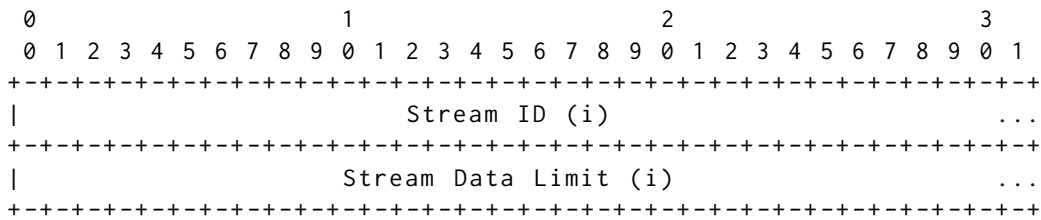
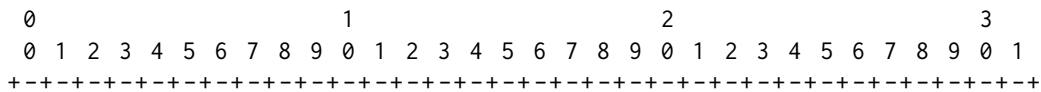


Figure 31: STREAM_DATA_BLOCKED Frame Format

5.1.11 STREAMS_BLOCKED Frames



```

|                               Stream Limit (i)                               ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 32: STREAMS_BLOCKED Frame Format

5.1.12 NEW_CONNECTION_ID Frame

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               Sequence Number (i)                               ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|   Length (8)   |                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               Connection ID (32..144)                               +
|                               ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               Stateless Reset Token (128)                               +
|                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 33: NEW_CONNECTION_ID Frame Format

5.1.13 RETIRE_CONNECTION_ID Frame

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               Sequence Number (i)                               ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 34: RETIRE_CONNECTION_ID Frame Format

5.1.14 PATH_CHALLENGE Frame

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               Data (64)                               +
|                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 35: PATH_CHALLENGE Frame Format

5.1.15 CONNECTION_CLOSE Frames

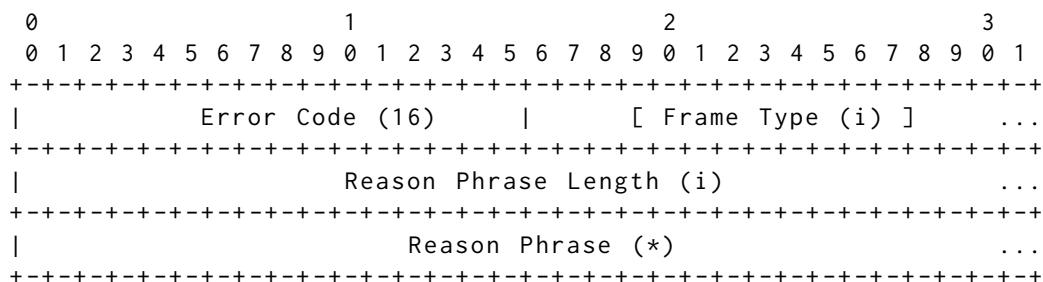


Figure 36: CONNECTION_CLOSE Frame Format

6 Transport Error Codes

QUIC error codes are 62-bit unsigned integers. This section lists the defined QUIC transport error codes that may be used in a CONNECTION_CLOSE frame. These errors apply to the entire connection.

NO_ERROR (0x0): An endpoint uses [this](#) with CONNECTION_CLOSE to signal that the connection is being closed abruptly in the absence of any error.

INTERNAL_ERROR (0x1): The endpoint encountered an internal error and cannot [continue](#) with the connection.

SERVER_BUSY (0x2): The server is currently busy and does not accept any [new](#) connections.

FLOW_CONTROL_ERROR (0x3): An endpoint received more data than it permitted in its advertised data limits (see Section 4).

STREAM_LIMIT_ERROR (0x4): An endpoint received a frame [for](#) a stream identifier that exceeded its advertised stream limit [for](#) the corresponding stream [type](#).

STREAM_STATE_ERROR (0x5): An endpoint received a frame [for](#) a stream that was not in a state that permitted that frame (see Section 3).

FINAL_SIZE_ERROR (0x6): An endpoint received a STREAM frame containing data that exceeded the previously established [final](#) size. Or an endpoint received a STREAM frame or a RESET_STREAM frame containing a [final](#) size that was lower than the size of stream data that was already received. Or an endpoint received a STREAM frame or a RESET_STREAM frame containing a different [final](#) size to the one already established.

FRAME_ENCODING_ERROR (0x7): An endpoint received a frame that was badly formatted. For instance, a frame of an unknown [type](#), or an ACK frame that has more acknowledgment ranges than the remainder

of the packet could carry.

TRANSPORT_PARAMETER_ERROR (0x8): An endpoint received transport parameters that were badly formatted, included an invalid value, was absent even though it is mandatory, was present though it is forbidden, or is otherwise in error.

VERSION_NEGOTIATION_ERROR (0x9): An endpoint received transport parameters that contained version negotiation parameters that disagreed with the version negotiation that it performed. This error code indicates a potential version downgrade attack.

PROTOCOL_VIOLATION (0xA): An endpoint detected an error with protocol compliance that was not covered by more specific error codes.

INVALID_MIGRATION (0xC): A peer has migrated to a different network when the endpoint had disabled migration.

CRYPTO_ERROR (0x1XX): The cryptographic handshake failed. A range of 256 values is reserved for carrying error codes specific to the cryptographic handshake that is used. Codes for errors occurring when TLS is used for the crypto handshake are described in Section 4.8 of [QUIC-TLS].

7 QUIC TLS

7.1 Carrying TLS Messages

Packet Type	Encryption Level	PN Space
Initial	Initial secrets	Initial
0-RTT Protected	0-RTT	0/1-RTT
Handshake	Handshake	Handshake
Retry	N/A	N/A
Short Header	1-RTT	0/1-RTT

Table 1: Encryption Levels by Packet Type

7.2 Header Protection

```
mask = header_protection(hp_key, sample)
```

```

pn_length = (packet[0] & 0x03) + 1
if (packet[0] & 0x80) == 0x80:
    # Long header: 4 bits masked
    packet[0] ^= mask[0] & 0x0f
else:
    # Short header: 5 bits masked
    packet[0] ^= mask[0] & 0x1f

# pn_offset is the start of the Packet Number field.
packet[pn_offset:pn_offset+pn_length] ^= mask[1:1+pn_length]

```

Figure 4: Header Protection Pseudocode

Figure 7 shows the **protected** fields of **long** and **short** headers marked with an E. Figure 7 also shows the sampled fields.

```

Long Header:
+++++
|1|1|T T|E E E E|
+++++
|                                     Version -> Length Fields ...
+++++

Short Header:
+++++
|0|1|S|E E E E E|
+++++
|                                     Destination Connection ID (0/32..144) ...
+++++

Common Fields:
+++++
|E E E E E E E E E Packet Number (8/16/24/32) E E E E E E E...
+++++
| [Protected Payload (8/16/24)] ...
+++++
|                                     Sampled part of Protected Payload (128) ...
+++++
|                                     Protected Payload Remainder (*) ...
+++++

```

Figure 5: Header Protection and Ciphertext Sample

The sampled ciphertext **for** a packet with a **short** header can be determined by the following pseudocode:

```

sample_offset = 1 + len(connection_id) + 4

sample = packet[sample_offset..sample_offset+sample_length]
sample_offset = 7 + len(destination_connection_id) +
                len(source_connection_id) +

```

```
        len(payload_length) + 4
if packet_type == Initial:
    sample_offset += len(token_length) +
                    len(token)

sample = packet[sample_offset..sample_offset+sample_length]
```